الجمهورية العربية السورية
وزارة التعليم العالي
الجامعة الافتراضية السورية

وظيفة مقرر BAIT_IWB303 — الفصل F25

تطبيق دليل الإسعافات الأولية للطوارئ (FirstAid Hero)

التقرير التأملي: دليل المبرمج ودليل المستخدم والمستثمر

البيان	التفاصيل
اسم الطالب	[اسم الطالب]
الرقم الجامعي	[الرقم الجامعي]
الشعبة	[رقم الشعبة C?]
أعضاء الفريق	[أسماء أعضاء الفريق وأرقامهم]
أستاذ المقرر	د. محمد مازن المصطفى
تاريخ التسليم	2026/11/5

سلم تصحيح الوظيفة

المعيار	الوصف	الدرجة	العلامة المستحقة
جودة وقوة الكود (Code Quality)	خلو الكود من الأخطاء، صحة تضمين الـ DB مسبقة الصنع بكفاءة، وسرعة استجابة استعلامات البحث الفوري	30	
عمليات CRUD والواجهات	عمل شريط البحث الديناميكي بكفاءة، إمكانية إضافة وتعديل أرقام الطوارئ المخصصة، وتناسق واجهات الطوارئ	20	
التقرير: دليل المبرمج	دقة الشرح التقني، وضوح الـ ERD، والتحليل التأملي للمشاكل البرمجية التي واجهت الفريق أثناء التطوير وكيفية التغلب عليها	25	
التقرير: دليل المستخدم/المستثمر	وضوح لقطات الشاشة الإرشادية، وقوة الحجة التسويقية والتحليلية لتجربة المستخدم في الظروف الحرجة	15	
النظام	معالجة الأخطاء واستقرار اختبار التطبيق ضد المدخلات الخاطئة (Input Validation) وضمان عدم انهياره (Zero Crashes)	10	
المجموع الكلي		100	

فهرس المحتويات

مقدمة 4
ملخص ما أنجزناه 4
القسم الأول: دليل المبرمج (Developer's Guide) 5
1. التقنيات وبيئة العمل 5
2. مخطط قاعدة البيانات (ERD) 5
2.1 الجداول 5
2.2 تبرير العلاقات 7
2.3 الفهارس 7
3. بنية المشروع (Architecture) 8
3.1 الفئات الرئيسية 9
3.2 قرارات التصميم 9
3.3 التنقل بين الشاشات 10
4. تضمين قاعدة البيانات الجاهزة (Pre-populated DB) 11
4.1 توليد ملف القاعدة 11
4.2 النسخ عند أول تشغيل 12
4.3 الإصدارات والترحيل دون فقدان البيانات 14
5. استعلامات SQL 17
5.1 البحث الفوري (LIKE و MATCH) 17
5.2 التقارير الإحصائية 19
5.3 سرعة الاستعلامات 20
6. عمليات CRUD 20
7. صورة الحالة المعالجة 22
8. الاتصال المباشر والصلاحيات 24
9. معالجة الأخطاء والتحقق من المدخلات 26
10. المشاكل البرمجية التي واجهتنا وكيف تغلبنا عليها 29
10.1 انهيار التطبيق مع أول حرف في البحث 29
10.2 البحث بالعربية لا يجد شيئاً 29
10.3 شريط التنقل السفلي يميّز التبويب الخطأ 30
10.4 تداخل المحتوى مع أشرطة النظام 30
10.5 الوظيفة تطلب قاعدة جاهزة بينما كنا نملأ البيانات من الكود 30
10.6 صورة الحالة كانت تُحفظ ولا تظهر 31
10.7 نموذج رقم الطوارئ يمسح ما كتبه المستخدم 31
10.8 رموز البحث % و _ تطابق كل شيء 31

10.9 الزوايا الدائرية شبه مربعة على الشاشات عالية الدقة 31
10.10 تثبيت ملف تطبيق قديم أثناء الاختبار 31
10.11 المسافات بين العناصر في الاتجاه العربي 31
10.12 فجوات اكتشفناها بمراجعة الوظيفة سطراً بسطر 32
11. الاختبار 32
القسم الثاني: دليل المستخدم والمستثمر (User / Investor Guide) 33
1. دليل التشغيل (User Manual) 33
1.1 التثبيت والتشغيل 33
1.2 البحث عن حالة طارئة 34
1.3 اتباع خطوات الإسعاف والاتصال السريع 35
1.4 إضافة رقم طوارئ جديد 37
1.5 تسجيل حالة معالجة 40
1.6 الإحصاءات والإعدادات 42
2. فلسفة تجربة المستخدم (UI/UX Justification) 45
3. جدوى المشروع (Value Proposition) 45
3.1 المشكلة 45
3.2 الحل 45
3.3 كيف ينقذ الأرواح 45
3.4 الفئات المستهدفة 46
3.5 الجدوى الاقتصادية 46
3.6 الحدود والمخاطر 46
الخاتمة والتأمل 47
ملحق: ملفات المشروع 47

مقدمة

يستعرض هذا التقرير التأملي مراحل بناء تطبيق **"دليل الإسعافات الأولية للطوارئ" (FirstAid Hero)**، وهو تطبيق أندرويد طبي خدمي يعمل دون اتصال بالإنترنت، ويقدم إرشادات إسعاف أولي سريعة وواضحة لحالات الطوارئ الشائعة مثل الحروق والكسور والغرق والنوبات القلبية.

انطلقنا في تصميم التطبيق من فكرة واحدة: في حالة الطوارئ قد يعرّض تأخر المعلومة لثوانٍ حياةَ إنسان للخطر. لذلك وضعنا ثلاثة أهداف حكمت كل قراراتنا: أن يعمل التطبيق في أي مكان دون إنترنت، وأن يصل المستخدم إلى الخطوات الصحيحة بأقل عدد من الضغطات، وأن تكون المعلومات مقروءة بوضوح تحت الضغط والتوتر.

ينقسم التقرير إلى قسمين كما تطلب الوظيفة:

- **القسم الأول: دليل المبرمج.** يشرح مخطط قاعدة البيانات وتبرير العلاقات، وبنية المشروع والفئات، وطريقة تضمين قاعدة البيانات الجاهزة ونسخها عند أول تشغيل، وأكواد SQL للبحث الفوري والإحصاءات، والمشاكل البرمجية التي واجهتنا وكيف تغلبنا عليها.
- **القسم الثاني: دليل المستخدم والمستثمر.** يشرح طريقة استخدام التطبيق بلقطات الشاشة، وفلسفة تصميم الواجهة، والجدوى العملية والاقتصادية للمشروع.

ملخص ما أنجزناه

المتطلب في الوظيفة	ما نُفّذ في التطبيق
تطبيق يعمل دون إنترنت	لا يطلب التطبيق إذن الإنترنت أصلاً، وكل البيانات على الجهاز
قاعدة بيانات SQLite جاهزة من مجلد assets	تُنسخ عند أول تشغيل `assets/databases/firstaid_hero.db`
بحث فوري بـ LIKE و MATCH	فهرس FTS4 مع `LIKE`، والنتائج تتحدث مع كل حرف
الجداول الأربعة المطلوبة	الحالات، الخطوات (واحد-إلى-متعدد)، أرقام الطوارئ، الحالات المعالجة
الشاشات المطلوبة	الرئيسية والبحث، خطوات الإسعاف، أرقام الطوارئ، إدخال الحالات المعالجة
الإحصاءات	تصنيف الحالات حسب الخطورة باستعلامات الفرز والتجميع، وعرض الحالات المعالجة حسب التاريخ مع البحث

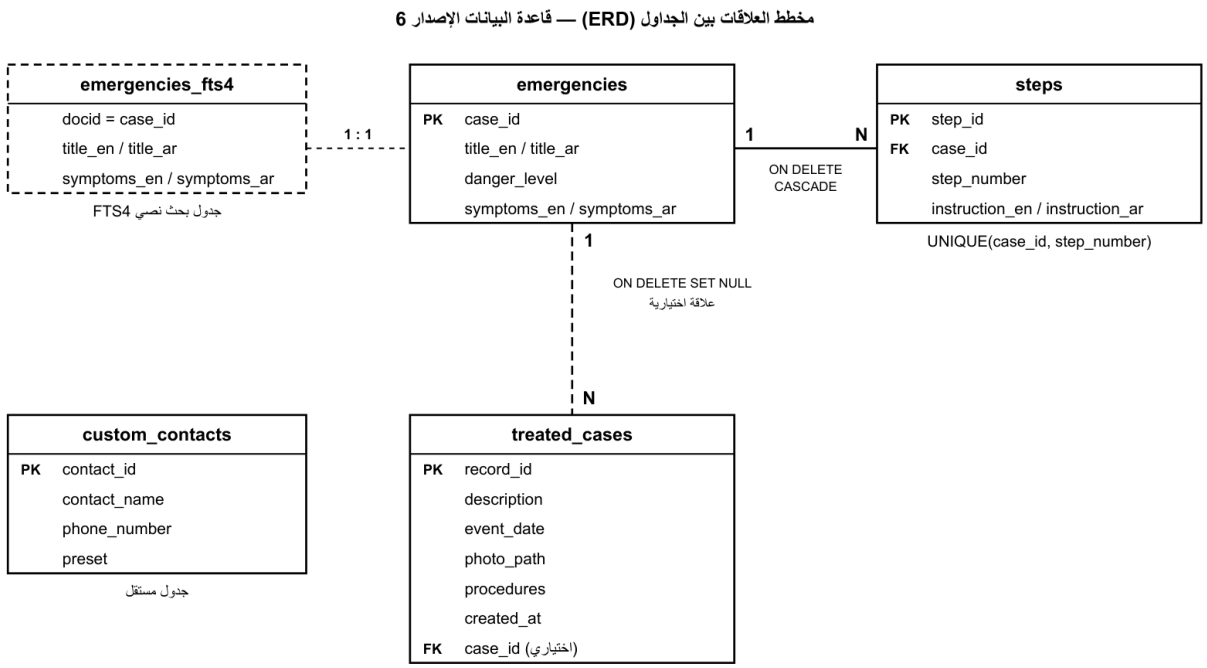
القسم الأول: دليل المبرمج (Developer's Guide)

1. التقنيات وبيئة العمل

العنصر	الاختيار	السبب
بيئة التطوير	Android Studio	البيئة المطلوبة في الوظيفة
لغة البرمجة	Java 11	اللغة المطلوبة في الوظيفة
أدنى إصدار أندرويد	Android 11 (API 30)	يغطي معظم الأجهزة الحالية ويدعم المكتبات الحديثة لمنتقي الصور وفك ترميز الصور
قاعدة البيانات	SQLite عبر `SQLiteOpenHelper`	مدمجة في أندرويد، سريعة، ولا تحتاج خادماً
البحث النصي	جدول افتراضي FTS4	متوفر في كل أجهزة أندرويد، على عكس FTS5
المكتبات	AndroidX AppCompat، AndroidX Activity، Material Components	التوافق بين الإصدارات، الصلاحيات ومنتقي الصور، وشريط التنقل السفلي
توليد القاعدة الجاهزة	سكربت Python	يولّد ملف القاعدة بالمحتوى العربي والإنجليزي بشكل قابل للتكرار

2. مخطط قاعدة البيانات (ERD)

يتكون مخطط قاعدة البيانات من الجداول الأربعة التي حددتها الوظيفة، إضافة إلى جدول بحث نصي افتراضي يسرّع البحث.



الشكل 1: مخطط العلاقات بين الجداول (ERD)

2.1 الجداول

جدول الحالات الإسعافية: `emergencies`

العمود	النوع والقيود	المعنى	الاسم في الوظيفة
case_id	INTEGER PRIMARY KEY	رقم الحالة	case_id
title_en / title_ar	TEXT NOT NULL	عنوان الحالة بالإنجليزية والعربية	title
danger_level	TEXT NOT NULL CHECK (critical / moderate / minor)	مستوى الخطورة: حرج، متوسط، طفيف	danger_level
symptoms_en / symptoms_ar	TEXT NOT NULL	الأعراض المصاحبة باللغتين	symptoms

جدول خطوات الإسعاف :steps

العمود	النوع والقيود	المعنى	الاسم في الوظيفة
step_id	INTEGER PRIMARY KEY AUTOINCREMENT	رقم الخطوة	step_id
case_id	مفتاح أجنبي إلى emergencies مع ON DELETE CASCADE	الحالة التي تتبع لها الخطوة	case_id
step_number	INTEGER NOT NULL	ترتيب الخطوة 1، 2، 3	step_number
instruction_en / instruction_ar	TEXT NOT NULL	نص الإجراء الإسعافي باللغتين	instruction_text

جدول أرقام الطوارئ :custom_contacts

العمود	النوع والقيود	المعنى
contact_id	INTEGER PRIMARY KEY AUTOINCREMENT	رقم السجل
contact_name	TEXT NOT NULL	الاسم
phone_number	TEXT NOT NULL	رقم الهاتف
preset	INTEGER DEFAULT 0	1 للأرقام العامة (شرطة، إسعاف، إطفاء) التي لا يمكن تعديلها أو حذفها

جدول الحالات المعالجة :treated_cases

العمود	النوع والقيود	المعنى
record_id	INTEGER PRIMARY KEY AUTOINCREMENT	رقم السجل
description	TEXT NOT NULL	وصف الحالة
event_date	TEXT NOT NULL بصيغة YYYY-MM-DD	تاريخ الحالة

العمود	النوع والقيود	المعنى
photo_path	TEXT (اختياري)	اسم ملف صورة الحالة المحفوظة داخل التطبيق
procedures	TEXT NOT NULL	الإجراءات التي أُجريت
created_at	INTEGER (تلقائي)	وقت إنشاء السجل
case_id	مفتاح أجنبي اختياري مع ON DELETE SET NULL	نوع الحالة الإسعافية المعالجة

2.2 تبرير العلاقات

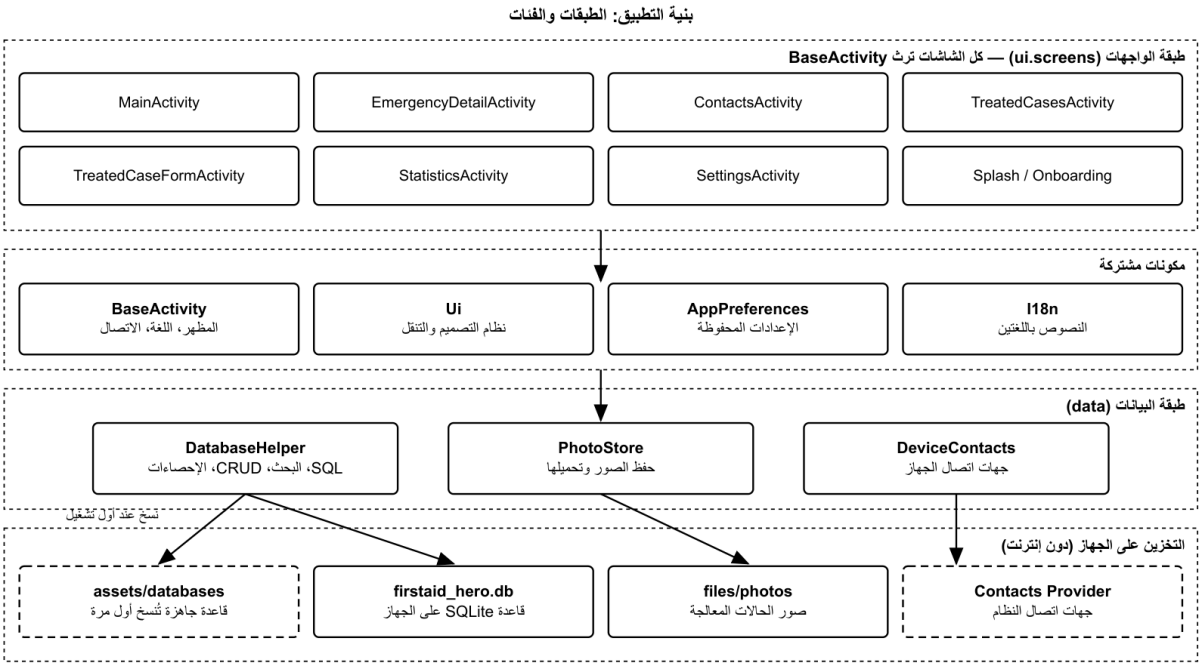
- **الحالات ← الخطوات (واحد-إلى-متعدد):** لكل حالة عدة خطوات مرتبة كما تطلب الوظيفة. فصلنا الخطوات في جدول مستقل بدل تخزينها نصاً واحداً، حتى نعرضها قائمة مرقمة ونضمن ترتيبها بالعمود step_number. القيد UNIQUE(case_id, step_number) يمنع تكرار رقم الخطوة في الحالة نفسها، وON DELETE CASCADE يمنع بقاء خطوات بلا حالة.
- **الحالات ← الحالات المعالجة (واحد-إلى-متعدد اختيارية):** ربطنا كل سجل معالجة بنوع الحالة الإسعافية حتى نستطيع إحصاء الحالات المعالجة حسب الخطورة. جعلنا الربط اختيارياً لأن المستخدم قد يعالج حالة غير موجودة في الدليل. اخترنا ON DELETE SET NULL حتى لا تُحذف سجلات المستخدم إذا تغيّر محتوى الدليل.
- **أرقام الطوارئ جدول مستقل:** لا يرتبط بغيره، والعمود preset يميّز الأرقام العامة عن أرقام المستخدم.
- **جدول البحث emergencies_fts4:** علاقة واحد-إلى-واحد مع الحالات (docid يساوي case_id)، ويُعاد بناؤه من جدول الحالات.
- **ثنائية اللغة:** قسمنا title و symptoms و instruction_text إلى عمودين (عربي وإنجليزي) لأن التطبيق يدعم اللغتين. هذا توسيع لمخطط الوظيفة دون تغيير معناه.

2.3 الفهارس

الفهرس	الغرض
idx_emergency_severity على danger_level	تسريع التجميع والترتيب حسب الخطورة
idx_treated_date على event_date	تسريع عرض السجل حسب التاريخ
idx_treated_case على case_id	تسريع الربط بين السجل والحالات في القائمة والإحصاءات

3. بنية المشروع (Architecture)

قسمنا الكود إلى ثلاث حزم بحسب المسؤولية: `core` للإعدادات والترجمة، و`data` للوصول إلى البيانات، و`ui` للواجهات. لا تتعامل أي شاشة مع ملف أو جدول مباشرة، بل عبر فئات الحزمة `data`.



الشكل 2: بنية التطبيق: الطبقات والفئات

app/src/main/java/com/firstaidhero/app

```
com.firstaidhero.app
├── core/
│   ├── AppPreferences.java          إعدادات اللغة والمظهر والتشغيل الأول
│   └── I18n.java                    نصوص الواجهة بالعربية والإنجليزية
├── data/
│   ├── DatabaseHelper.java          قاعدة البيانات والبحث و CRUD والإحصاءات
│   ├── DeviceContacts.java          قراءة جهات اتصال الجهاز
│   └── PhotoStore.java              حفظ صور الحالات المعالجة
└── ui/
    ├── Ui.java                      نظام التصميم المشترك وشريط التنقل
    └── screens/
        ├── BaseActivity.java        الأب المشترك: المظهر، اللغة، الاتصال
        ├── SplashActivity.java      شاشة البداية
        ├── OnboardingActivity.java  شاشات التعريف
        ├── MainActivity.java        الرئيسية والبحث
        ├── EmergencyDetailActivity.java  خطوات الإسعاف وزر الاتصال
        ├── ContactsActivity.java    أرقام الطوارئ
        ├── TreatedCasesActivity.java  سجل الحالات المعالجة
        ├── TreatedCaseFormActivity.java  إدخال حالة معالجة
        ├── StatisticsActivity.java  الإحصاءات
        └── SettingsActivity.java    الإعدادات
```

الشكل 3: هيكل الحزم والفئات في المشروع

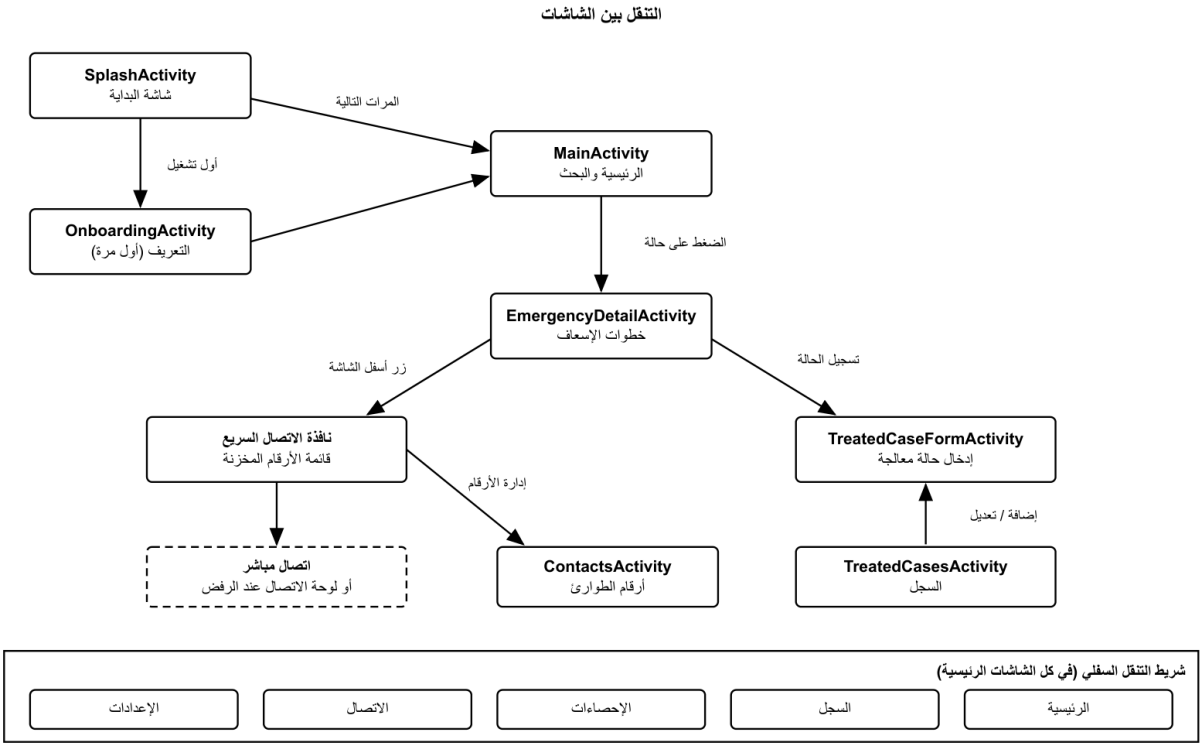
3.1 الفئات الرئيسية

الفئة	المسؤولية
`DatabaseHelper`	يرث `SQLiteOpenHelper`: نسخ القاعدة الجاهزة، إنشاء الجداول، الترحيل، البحث، CRUD، والإحصاءات
`PhotoStore`	حفظ صور الحالات المعالجة داخل التطبيق وتحميلها بالحجم المناسب
`DeviceContacts`	قراءة جهات اتصال الجهاز لاختيار رقم طوارئ منها
`AppPreferences`	حفظ اللغة والمظهر وإتمام شاشات التعريف
`I18n`	كل نصوص الواجهة بالعربية والإنجليزية
`Ui`	نظام التصميم المشترك: الألوان والنصوص والأزرار وهيكل الشاشة وشريط التنقل
`BaseActivity`	الأب المشترك لكل الشاشات: المظهر، تحديث اللغة، والاتصال المباشر
`MainActivity`	الشاشة الرئيسية والبحث الفوري
`EmergencyDetailActivity`	شاشة خطوات الإسعاف وزر الاتصال السريع
`ContactsActivity`	شاشة أرقام الطوارئ (إضافة، تعديل، حذف، اتصال)
`TreatedCasesActivity`	سجل الحالات المعالجة مع البحث
`TreatedCaseFormActivity`	إدخال حالة معالجة وتعديلها مع الصورة
`StatisticsActivity`	الإحصاءات حسب الخطورة والتاريخ
`SettingsActivity` / `SplashActivity` / `OnboardingActivity`	الإعدادات، شاشة البداية، وشاشات التعريف

3.2 قرارات التصميم

- **بناء الواجهات برمجياً بلغة Java عبر الفئة `Ui`:** يضمن أن تكون كل الشاشات بالشكل نفسه (الألوان، الخطوط، الزوايا، الاتجاه من اليمين لليسار)، ويسهّل دعم الوضع الليلي واللغتين من مكان واحد.
- **شاشة لكل قسم (Activity):** كل تبويب في شريط التنقل السفلي شاشة مستقلة، ويُعاد استخدامها عند الرجوع إليها بالعلم `FLAG_ACTIVITY_REORDER_TO_FRONT`.
- **Adapters و ViewModels:** ذكرتهما الوظيفة أمثلةً على الفئات. لم نستخدمهما لأن حجم البيانات صغير (عشرات السجلات)، فبنينا القوائم مباشرة من نتيجة الاستعلام (`Cursor`) داخل كل شاشة، مع إغلاق المؤشر دائماً في `finally`. هذا أبسط وأسرع لهذا الحجم. إذا كبرت البيانات مستقبلاً يمكن الانتقال إلى `RecyclerView` مع `Adapter`.
- **التوثيق داخل الكود:** وضعنا تعليقاً عربياً فوق كل فئة وكل طريقة وبجانب كل مكتبة مستوردة.

3.3 التنقل بين الشاشات



الشكل 4: التنقل بين الشاشات

4. تضمين قاعدة البيانات الجاهزة (Pre-populated DB)

تطلب الوظيفة تضمين قاعدة بيانات مسبقة الصنع داخل التطبيق ونسخها من مجلد assets عند الإقلاع الأول. نفّذنا ذلك على ثلاث مراحل: توليد ملف القاعدة، شحنه داخل التطبيق، ثم نسخه وفتحه على الجهاز.

4.1 توليد ملف القاعدة

بدل إنشاء ملف القاعدة يدوياً، كتبنا سكربت Python يولّده بالكامل: الجداول، الحالات الثماني، الخطوات المرقمة، الأرقام العامة، وفهرس البحث. ميزة ذلك أن أي تعديل على المحتوى يُعاد توليده بأمر واحد، وأن المخطط في السكربت مطابق للمخطط في الكود.

tools/generate_assets_db.py

```
# أوامر إنشاء الجداول والفهارس وجدول البحث النصي FTS4 (مطابقة لـ onCreate في DatabaseHelper.)
SCHEMA = [
    """CREATE TABLE emergencies (
        case_id INTEGER PRIMARY KEY,
        title_en TEXT NOT NULL,
        title_ar TEXT NOT NULL,
        danger_level TEXT NOT NULL CHECK(danger_level IN ('critical','moderate','minor')),
        symptoms_en TEXT NOT NULL,
        symptoms_ar TEXT NOT NULL)""",
    """CREATE TABLE steps (
        step_id INTEGER PRIMARY KEY AUTOINCREMENT,
        case_id INTEGER NOT NULL,
        step_number INTEGER NOT NULL,
        instruction_en TEXT NOT NULL,
        instruction_ar TEXT NOT NULL,
        FOREIGN KEY(case_id) REFERENCES emergencies(case_id) ON DELETE CASCADE,
        UNIQUE(case_id, step_number))""",
    """CREATE TABLE custom_contacts (
        contact_id INTEGER PRIMARY KEY AUTOINCREMENT,
        contact_name TEXT NOT NULL,
        phone_number TEXT NOT NULL,
        preset INTEGER NOT NULL DEFAULT 0)""",
    """CREATE TABLE treated_cases (
        record_id INTEGER PRIMARY KEY AUTOINCREMENT,
        description TEXT NOT NULL,
        event_date TEXT NOT NULL,
        photo_path TEXT,
        procedures TEXT NOT NULL,
        created_at INTEGER NOT NULL DEFAULT (strftime('%s','now')),
        case_id INTEGER REFERENCES emergencies(case_id) ON DELETE SET NULL)""",
    "CREATE INDEX idx_emergency_severity ON emergencies(danger_level)",
    "CREATE INDEX idx_treated_date ON treated_cases(event_date)",
    "CREATE INDEX idx_treated_case ON treated_cases(case_id)",
    "CREATE VIRTUAL TABLE emergencies_fts4 USING fts4(title_en, title_ar, symptoms_en, symptoms_ar,
tokenize=unicode61)",
]
```

الشكل 5: أوامر إنشاء الجداول في سكربت التوليد

tools/generate_assets_db.py

```
def main():
    # تنشئ ملف القاعدة من جديد: الجداول، الحالات، فهرس البحث، الخطوات المرقمة، الأرقام العامة،
    # ثم تضبط رقم الإصدار user_version ليطابق DB_VERSION في التطبيق.
    os.makedirs(os.path.dirname(OUT), exist_ok=True)
    if os.path.exists(OUT):
        os.remove(OUT)
    con = sqlite3.connect(OUT)
    cur = con.cursor()
    for stmt in SCHEMA:
        cur.execute(stmt)
    for case_id, title_en, title_ar, level, symptoms_en, symptoms_ar, steps in CASES:
        cur.execute("INSERT INTO emergencies VALUES (?,?,?,?,?,?)", (case_id, title_en, title_ar,
level, symptoms_en, symptoms_ar))
        cur.execute("INSERT INTO emergencies_fts4(docid,title_en,title_ar,symptoms_en,symptoms_ar)
VALUES (?,?,?,?,?)", (case_id, title_en, title_ar, symptoms_en, symptoms_ar))
        for number, (step_en, step_ar) in enumerate(steps, start=1):
            cur.execute("INSERT INTO steps(case_id,step_number,instruction_en,instruction_ar) VALUES
(?,?,?,?)", (case_id, number, step_en, step_ar))
    for name, phone in PRESET_CONTACTS:
        cur.execute("INSERT INTO custom_contacts(contact_name,phone_number,preset) VALUES (?,?,1)",
(name, phone))
    cur.execute("PRAGMA user_version = 6")
    con.commit()
    con.close()
    print("Wrote", OUT)
```

الشكل 6: الدالة main في سكربت التوليد: تعبئة القاعدة وضبط رقم الإصدار

يُحفظ الملف الناتج في app/src/main/assets/databases/firstaid_hero.db، فيُضمَّن تلقائياً داخل ملف التطبيق عند البناء. ويضبط السكربت PRAGMA user_version = 6 ليطابق رقم الإصدار في الكود.

4.2 النسخ عند أول تشغيل

DatabaseHelper.java

```
/**
 * :(SQLiteOpenHelper برث) SQLite مدير قاعدة البيانات المحلية
 * - ينسخ قاعدة البيانات الجاهزة من مجلد assets عند أول تشغيل.
 * - ينشئ الجداول ويرحّل الإصدارات القديمة دون حذف بيانات المستخدم.
 * - يحتوي كل استعلامات البحث (LIKE و MATCH) وعمليات CRUD والإحصاءات.
 */
public class DatabaseHelper extends SQLiteOpenHelper {
    /** اسم ملف القاعدة على الجهاز، ومسارها داخل assets، ورقم إصدار المخطط الحالي. */
    private static final String DB_NAME = "firstaid_hero.db";
    private static final String ASSET_PATH = "databases/firstaid_hero.db";
    private static final int DB_VERSION = 6;
```

الشكل 7: ثوابت القاعدة في DatabaseHelper

عند فتح أي شاشة يُنشأ كائن DatabaseHelper، فيستدعي copyDatabaseFromAssets(). إذا لم يكن ملف القاعدة موجوداً على الجهاز، تُنسخ القاعدة الجاهزة من assets إلى مسار قواعد بيانات التطبيق على دفعات من 8 كيلوبايت. بعد ذلك تقرأ مكتبة SQLiteOpenHelper رقم الإصدار من الملف المنسوخ، فتجده مساوياً لـ DB_VERSION ولا تستدعي onCreate ولا onUpgrade.

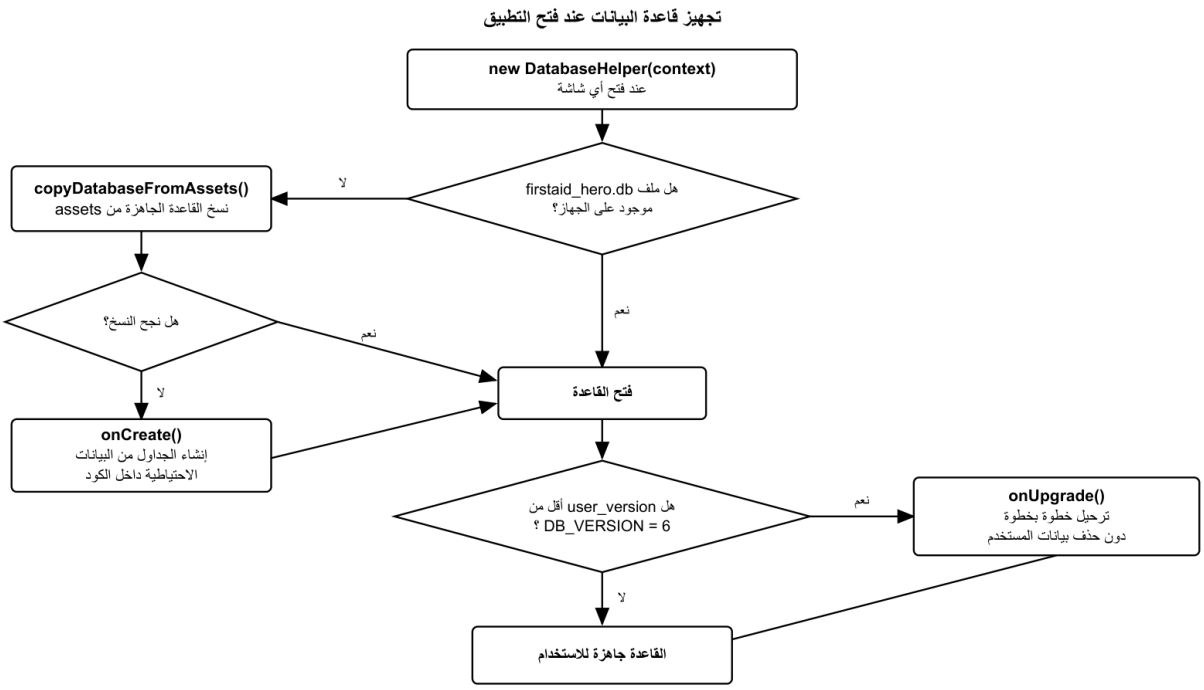
DatabaseHelper.java

```
/** المُنشئ: يحدد اسم القاعدة وإصدارها، ثم ينسخ القاعدة الجاهزة من assets إن لم تكن موجودة. */
public DatabaseHelper(Context context) {
    super(context, DB_NAME, null, DB_VERSION);
    copyDatabaseFromAssets(context);
}

/**
 * تنسخ القاعدة الجاهزة من assets/databases إلى مسار قواعد بيانات التطبيق عند أول تشغيل فقط.
 * عند الفشل تحذف أي ملف ناقص، فيُنشئ onCreate القاعدة من البيانات الاحتياطية.
 */
private void copyDatabaseFromAssets(Context context) {
    File target = context.getDatabasePath(DB_NAME);
    if (target.exists()) return;
    File parent = target.getParentFile();
    if (parent != null && !parent.exists() && !parent.mkdirs()) return;
    try (InputStream in = context.getAssets().open(ASSET_PATH);
         OutputStream out = new FileOutputStream(target)) {
        byte[] buffer = new byte[8192];
        int read;
        while ((read = in.read(buffer)) != -1) out.write(buffer, 0, read);
        out.flush();
    } catch (IOException e) {
        if (target.exists()) target.delete();
    }
}

/** تُستدعى عند كل فتح للقاعدة: تفعّل قيود المفاتيح الأجنبية (FOREIGN KEY). */
@Override public void onConfigure(SQLiteDatabase db) {
    super.onConfigure(db);
    db.setForeignKeyConstraintsEnabled(true);
}
```

الشكل 8: نسخ القاعدة الجاهزة من assets وتفعيل المفاتيح الأجنبية



الشكل 9: تجهيز قاعدة البيانات عند فتح التطبيق

حماية من الفشل: إذا تعذّر النسخ (ملف مفقود أو تالف) يحذف الكود أي ملف ناقص، فتُنشئ `onCreate` القاعدة نفسها من نسخة احتياطية من المحتوى موجودة داخل الكود (`SEED_CASES` و `SEED_STEPS`). بذلك لا يفتح المستخدم التطبيق أبداً على قاعدة فارغة.

4.3 الإصدارات والترحيل دون فقدان البيانات

مرّت القاعدة بستة إصدارات أثناء التطوير. المستخدم الذي ثبّت إصداراً قديماً لديه بيانات خاصة (أرقام وسجلات وصور) لا يجوز حذفها، لذلك ترحّل `onUpgrade` القاعدة خطوة بخطوة:

الإصدار	التغيير
1	المخطط الأول
2	عمود `preset` لحماية الأرقام العامة
3	فهرس البحث FTS4
4	إصلاح نصوص عربية تالفة في الملف الجاهز
5	حفظ الصورة كملف داخل التطبيق وربط السجل بنوع الحالة
6	إضافة حالتي الكسور والجروح البسيطة وتحديث حالة اللدغات

DatabaseHelper.java

```
/**
 * ترحّل قاعدة إصدار قديم إلى الإصدار الحالي خطوة بخطوة دون حذف بيانات المستخدم:
 * v2 حفظ الصورة كملف وربط السجل بنوع الحالة، v6 تحديث المحتوى وإعادة بناء فهرس البحث. عمود preset، v5
 */
@Override public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    if (oldVersion < 2) {
        try { db.execSQL("ALTER TABLE custom_contacts ADD COLUMN preset INTEGER NOT NULL DEFAULT
0"); } catch (SQLException ignored) { }
        seedPresetContacts(db);
    }
    if (oldVersion < 5) {
        db.execSQL("ALTER TABLE treated_cases RENAME COLUMN photo_uri TO photo_path");
        db.execSQL("ALTER TABLE treated_cases ADD COLUMN case_id INTEGER REFERENCES
emergencies(case_id) ON DELETE SET NULL");
        db.execSQL("CREATE INDEX IF NOT EXISTS idx_treated_case ON treated_cases(case_id)");
    }
    if (oldVersion < 6) {
        seedEmergencyContent(db);
        rebuildFts(db);
    }
}

/**
 * تحذف وتعيد إنشاء جدول البحث النصي FTS4 وتملؤه من جدول الحالات.
 * إذا لم يدعم الجهاز FTS4 يُتجاهل الخطأ ويعمل البحث بـ LIKE فقط.
 */
private void rebuildFts(SQLiteDatabase db) {
    try {
        db.execSQL("DROP TABLE IF EXISTS emergencies_fts4");
        db.execSQL("CREATE VIRTUAL TABLE emergencies_fts4 USING fts4(title_en, title_ar,
symptoms_en, symptoms_ar, tokenize=unicode61)");
        db.execSQL("INSERT INTO emergencies_fts4(docid, title_en, title_ar, symptoms_en,
symptoms_ar) SELECT case_id, title_en, title_ar, symptoms_en, symptoms_ar FROM emergencies");
    } catch (SQLiteException e) {
    }
}
```

الشكل 10: ترحيل الإصدارات وإعادة بناء فهرس البحث

درس تعلمناه: في البداية كان تحديث المحتوى يحذف الحالات ثم يعيد إدخالها. اكتشفنا أن الحذف سيقطع روابط السجلات المعالجة بالحالات (بسبب `ON DELETE SET NULL`)، فاستبدلنا ذلك بأمر **UPSERT** يحدّث الحالة الموجودة في مكانها:

DatabaseHelper.java

```
/**
 * تُدخل الحالات بأمر UPSERT: الحالة الجديدة تُضاف، والموجودة يُحدَّث نصها دون حذفها فلا تنقطع روابط السجلات بها.
 * الخطوات تُستبدل حسب المفتاح الفريد (case_id, step_number).
 */
private void seedEmergencyContent(SQLiteDatabase db) {
    for (int i = 0; i < SEED_CASES.length; i++) {
        String[] c = SEED_CASES[i];
        db.execSQL("INSERT INTO
emergencies(case_id,title_en,title_ar,danger_level,symptoms_en,symptoms_ar) VALUES (?,?,?,?,?,?) "
                + "ON CONFLICT(case_id) DO UPDATE SET title_en=excluded.title_en,
title_ar=excluded.title_ar, danger_level=excluded.danger_level, symptoms_en=excluded.symptoms_en,
symptoms_ar=excluded.symptoms_ar",
                new Object[]{Long.parseLong(c[0]), c[1], c[2], c[3], c[4], c[5]});
        String[][] steps = SEED_STEPS[i];
        for (int s = 0; s < steps.length; s++) {
            ContentValues step = new ContentValues(); step.put("case_id", Long.parseLong(c[0]));
            step.put("step_number", s + 1); step.put("instruction_en", steps[s][0]);
step.put("instruction_ar", steps[s][1]);
            db.insertWithOnConflict("steps", null, step, SQLiteDatabase.CONFLICT_REPLACE);
        }
    }
}
```

الشكل 11: تحديث المحتوى بأمر UPSERT دون حذف الحالات

5. استعلامات SQL

5.1 البحث الفوري (LIKE و MATCH)

البحث هو أهم وظيفة في التطبيق، لأن المستخدم في حالة طوارئ يكتب كلمة ناقصة ويريد النتيجة فوراً. لذلك يُنفَّذ البحث مع كل حرف يُكتب:

MainActivity.java

```
search.addTextChangedListener(new TextWatcher() {
    /** قبل تغيّر النص: لا نحتاجها. */
    @Override public void beforeTextChanged(CharSequence s, int start, int count, int after) { }
    /** مع كل حرف يُكتب أو يُحذف: تنفيذ البحث فوراً. */
    @Override public void onTextChanged(CharSequence s, int start, int before, int count) {
loadCases(s.toString()); }
    /** بعد تغيّر النص: لا نحتاجها. */
    @Override public void afterTextChanged(Editable e) { }
});
```

الشكل 12: ربط البحث بكل حرف يكتبه المستخدم في MainActivity

ويجمع الاستعلام طريقتين:

- **`MATCH` على فهرس FTS4:** يبحث ببادئات الكلمات وبأي ترتيب. مثلاً كتابة `sna ins` تجد "Snake or insect bite"، وهي نتيجة لا يجدها `LIKE` وحده.
- **`LIKE` على العناوين والأعراض باللغتين:** يجد أي جزء من الكلمة، مثل "لدغ" داخل "لدغات الأفاعي والحشرات" كما في مثال الوظيفة.

DatabaseHelper.java

```
/**
 * البحث الفوري: النص الفارغ يعيد كل الحالات، وإلا يُجمع MATCH (فهرس FTS4 لبدايات الكلمات)
 * مع LIKE (أي جزء من الكلمة) على العناوين والأعراض باللغتين، والنتائج مرتبة حسب الخطورة.
 * إذا فشل FTS على الجهاز يُعاد البحث بـ LIKE فقط حتى لا ينهار التطبيق.
 */
public Cursor searchEmergencies(String query) {
    SQLiteDatabase db = getReadableDatabase();
    String q = query == null ? "" : query.trim();
    String order = " ORDER BY " + SEVERITY_ORDER + ", title_en";
    String base = "SELECT case_id,title_en,title_ar,danger_level,symptoms_en,symptoms_ar FROM
emergencies";
    if (q.isEmpty()) return db.rawQuery(base + order, null);
    String pattern = likePattern(q);
    String match = buildMatchExpression(q);
    String likeClause = "title_en LIKE ? ESCAPE '\\' OR title_ar LIKE ? ESCAPE '\\' OR symptoms_en
LIKE ? ESCAPE '\\' OR symptoms_ar LIKE ? ESCAPE '\\'";
    try {
        String sql = base + " WHERE " +
                (match != null ? "case_id IN (SELECT docid FROM emergencies_fts4 WHERE
emergencies_fts4 MATCH ?) OR " : "") +
                likeClause + order;
        if (match != null) return db.rawQuery(sql, new String[]{match, pattern, pattern, pattern,
pattern});
        return db.rawQuery(sql, new String[]{pattern, pattern, pattern, pattern});
    } catch (SQLiteException e) {
        return db.rawQuery(base + " WHERE " + likeClause + order, new String[]{pattern, pattern,
pattern, pattern});
    }
}
```

الشكل 13: طريقة البحث الفوري searchEmergencies

تُرتَّب النتائج حسب الخطورة (حرج ثم متوسط ثم طفيف). إذا لم يدعم الجهاز FTS4 يلتقط الكود الخطأ ويعيد البحث بـ LIKE وحده، فلا ينهار التطبيق.

DatabaseHelper.java

```
/**
 * .(*sna ins ← sna* ins) تحوّل نص البحث إلى تعبير FTS ببادئة لكل كلمة
 * .FTS تعيد null إذا احتوى النص رموزاً غير آمنة لـ
 */
private String buildMatchExpression(String q) {
    if (!q.matches("[\\p{L}\\p{N}][\\p{L}\\p{N}\\s]*")) return null;
    String[] tokens = q.split("\\s+");
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < tokens.length; i++) {
        if (i > 0) sb.append(' ');
        sb.append(tokens[i]).append('*');
    }
    return sb.toString();
}

/** تحيط النص بـ %...% وتهرّب المحارف % و _ و \ حتى تُعامل كنص عادي وليس كرموز بحث. */
private static String likePattern(String q) {
    return "%" + q.replace("\\", "\\\\").replace("%", "\\%").replace("_", "\\_") + "%";
}
```

الشكل 14: بناء تعبير MATCH وتهريب رموز LIKE

الأمان: كل القيم تمرَّر كمعاملات ? وليس بدمج النص في الاستعلام، فلا مجال لحقن SQL. والدالة `likePattern` تهرّب الرموز % و _ و \ حتى لا يطابق البحث كل شيء إذا كتبها المستخدم.

5.2 التقارير الإحصائية

تطلب الوظيفة شاشة تعرض تصنيف الحالات حسب خطورتها باستعلامات الفرز والمجاميع التكرارية. استخدمنا `GROUP BY` مع `COUNT(*)` و `ORDER BY` حسب الخطورة:

DatabaseHelper.java

```
/** إحصاء: عدد حالات الدليل لكل مستوى خطورة (GROUP BY + COUNT + ORDER BY). */
public Cursor countEmergenciesBySeverity() {
    return getReadableDatabase().rawQuery("SELECT danger_level, COUNT(*) FROM emergencies GROUP BY
danger_level ORDER BY " + SEVERITY_ORDER, null);
}

/** إحصاء: عدد الحالات المعالجة لكل مستوى خطورة؛ السجل غير المرتبط بحالة يُحسب unspecified. */
public Cursor countTreatedBySeverity() {
    return getReadableDatabase().rawQuery("SELECT COALESCE(e.danger_level,'unspecified') AS
danger_level, COUNT(*) FROM treated_cases t LEFT JOIN emergencies e ON e.case_id=t.case_id GROUP BY
1 ORDER BY " + SEVERITY_ORDER, null);
}

/** إحصاء: عدد الحالات المعالجة في كل شهر (آخر ستة أشهر فيها سجلات). */
public Cursor countTreatedByMonth() {
    return getReadableDatabase().rawQuery("SELECT substr(event_date,1,7) AS month, COUNT(*) FROM
treated_cases GROUP BY month ORDER BY month DESC LIMIT 6", null);
}
```

الشكل 15: استعلامات الإحصاء: حسب الخطورة وحسب الشهر

النتيجة على المحتوى الحالي: 4 حالات حرجة، و3 متوسطة، وحالة واحدة طفيفة. وتعرض الشاشة النتائج أشرطةً أفقية:

StatisticsActivity.java

```
/** تنفذ الاستعلامات التجميعية الثلاثة وتعرض ثلاث بطاقات. */
private void load() {
    sections.removeAllViews();
    Map<String, Integer> guide = zeroed(LEVELS);
    readCounts(db.countEmergenciesBySeverity(), guide);
    addSection(I18n.t(this, "stats_guide"), labelled(guide));

    Map<String, Integer> treated = zeroed(LEVELS); treated.put("unspecified", 0);
    readCounts(db.countTreatedBySeverity(), treated);
    addSection(I18n.t(this, "stats_treated"), labelled(treated));

    Map<String, Integer> months = new LinkedHashMap<>();
    readCounts(db.countTreatedByMonth(), months);
    addSection(I18n.t(this, "stats_months"), months);
}

/** تقرأ نتيجة GROUP BY إلى خريطة (المستوى ← العدد). */
private static void readCounts(Cursor c, Map<String, Integer> into) {
    try { while (c.moveToNext()) if (!c.isNull(0)) into.put(c.getString(0), c.getInt(1)); } finally
{ c.close(); }
}
```

الشكل 16: قراءة نتائج التجميع في شاشة الإحصاءات

5.3 سرعة الاستعلامات

قسنا زمن الاستعلامات الفعلية على القاعدة الجاهزة (200 تنفيذ تمهيدي ثم 20,000 تنفيذ لكل استعلام):

الاستعلام	النمط	متوسط الزمن	النتائج
بحث فارغ (كل الحالات)	مسح كامل	130 ميكروثانية	8
"bur"	MATCH + LIKE	194 ميكروثانية	1
"sna ins"	MATCH + LIKE	205 ميكروثانية	1
"لدغ"	MATCH + LIKE	200 ميكروثانية	1
"كسو"	MATCH + LIKE	206 ميكروثانية	1
إحصاء الحالات حسب الخطورة	GROUP BY + COUNT	116 ميكروثانية	3

كل استعلام أقل من ربع ملّي ثانية، فالبحث فوري فعلاً ويمكن تنفيذه مع كل ضغطة مفتاح دون أي تأخير.

6. عمليات CRUD

البيانات	إضافة / تعديل	قراءة	حذف
الحالات والخطوات	محتوى جاهز للقراءة فقط	`searchEmergencies`، `getEmergency`، `getSteps`	—

البيانات	إضافة / تعديل	قراءة	حذف
أرقام الطوارئ	saveContact	getContacts	deleteContact
الحالات المعالجة	saveTreatedCase	getTreatedCase، getTreatedCases	deleteTreatedCase

DatabaseHelper.java

```
/**
 * تضيف رقماً جديداً إذا كان id سالباً، وإلا تعدّله. الأرقام العامة محمية بالشرط preset = 0.
 * تعيد رقم السجل، أو -1 عند الفشل.
 */
public long saveContact(long id, String name, String phone) {
    ContentValues v = new ContentValues(); v.put("contact_name", name.trim()); v.put("phone_number",
phone.trim());
    SQLiteDatabase db = getWritableDatabase();
    if (id < 0) return db.insert("custom_contacts", null, v);
    return db.update("custom_contacts", v, "contact_id=? AND preset=0", new String[]
{String.valueOf(id)}) > 0 ? id : -1;
}

/** تجلب كل الأرقام: العامة أولاً ثم أرقام المستخدم أبجدياً. */
public Cursor getContacts() { return getReadableDatabase().rawQuery("SELECT
contact_id,contact_name,phone_number,preset FROM custom_contacts ORDER BY preset DESC, contact_name
COLLATE NOCASE", null); }

/** تحذف رقماً أضافه المستخدم (الأرقام العامة لا تُحذف). */
public int deleteContact(long id) { return getWritableDatabase().delete("custom_contacts",
"contact_id=? AND preset=0", new String[]{String.valueOf(id)}); }
```

الشكل 17: عمليات CRUD لأرقام الطوارئ

الأرقام العامة محمية من التعديل والحذف بالشرط `preset = 0` داخل الاستعلام نفسه، وليس في الواجهة فقط.

DatabaseHelper.java

```
/**
 * تضيف سجل حالة معالجة (id سالب) أو تعدّله.
 * نوع الحالة الإسعافية وقد يكون null، و photoPath اسم ملف الصورة داخل التطبيق. caseId
 */
public long saveTreatedCase(long id, Long caseId, String description, String date, String photoPath,
String procedures) {
    ContentValues v = new ContentValues(); v.put("description", description.trim());
v.put("event_date", date.trim());
    v.put("photo_path", photoPath); v.put("procedures", procedures.trim());
    if (caseId == null) v.putNull("case_id"); else v.put("case_id", caseId);
    SQLiteDatabase db = getWritableDatabase();
    if (id < 0) return db.insert("treated_cases", null, v);
    return db.update("treated_cases", v, "record_id=?", new String[]{String.valueOf(id)}) > 0 ? id :
-1;
}

/** بداية استعلام السجل مع LEFT JOIN على جدول الحالات لجلب عنوان الحالة المرتبطة وخطورتها. */
private static final String TREATED_COLUMNS = "SELECT
t.record_id,t.description,t.event_date,t.photo_path,t.procedures,t.case_id,e.title_en,e.title_ar,e.d
anger_level FROM treated_cases t LEFT JOIN emergencies e ON e.case_id=t.case_id";

/** تجلب السجلات من الأحدث للأقدم، مع بحث في الوصف والإجراءات والتاريخ وعنوان الحالة. */
public Cursor getTreatedCases(String query) {
    String q = query == null ? "" : query.trim();
    String sql = TREATED_COLUMNS;
    if (!q.isEmpty()) sql += " WHERE t.description LIKE ? ESCAPE '\\' OR t.procedures LIKE ? ESCAPE
'\\' OR t.event_date LIKE ? ESCAPE '\\' OR e.title_en LIKE ? ESCAPE '\\' OR e.title_ar LIKE ? ESCAPE
'\\'";
    sql += " ORDER BY t.event_date DESC, t.record_id DESC";
    if (q.isEmpty()) return getReadableDatabase().rawQuery(sql, null);
    String pattern = likePattern(q);
    return getReadableDatabase().rawQuery(sql, new String[]{pattern, pattern, pattern, pattern,
pattern});
}

/** تحذف سجل حالة معالجة. */
public int deleteTreatedCase(long id) { return getWritableDatabase().delete("treated_cases",
"record_id=?", new String[]{String.valueOf(id)}); }
```

الشكل 18: عمليات CRUD للحالات المعالجة مع LEFT JOIN

7. صورة الحالة المعالجة

تطلب الوظيفة حفظ صورة للحالة المعالجة. فكرنا في حلين: تخزين الصورة داخل القاعدة (BLOB)، أو نسخها إلى ملف داخل التطبيق وتخزين اسمه فقط. اخترنا الحل الثاني لأن أندرويد يقرأ الصف الواحد من القاعدة بحد أقصى نحو 2 ميغابايت، ولأن الصور الكبيرة داخل القاعدة تبطئ كل استعلام على القائمة.

PhotoStore.java

```
/**
 * تفك ترميز الصورة المختارة (مع تصحيح اتجاهها حسب EXIF)، وتصغّرها إلى 1280 بكسل كحد أقصى،
 * ثم تحفظها بصيغة JPEG باسم فريد وتعيد اسم الملف. عند الفشل تحذف الملف الناقص وترمي استثناء.
 */
public static String importPhoto(Context context, Uri source) throws IOException {
    Bitmap bitmap =
ImageDecoder.decodeBitmap(ImageDecoder.createSource(context.getContentResolver(), source), (decoder,
info, src) -> {
        int w = info.getSize().getWidth(), h = info.getSize().getHeight();
        float scale = Math.min(1f, MAX_EDGE_PX / (float) Math.max(w, h));
        decoder.setTargetSize(Math.max(1, Math.round(w * scale)), Math.max(1, Math.round(h *
scale)));
        decoder.setAllocator(ImageDecoder.ALLOCATOR_SOFTWARE);
    });
    File folder = dir(context);
    if (!folder.exists() && !folder.mkdirs()) throw new IOException("Cannot create " + folder);
    String name = "case_" + System.currentTimeMillis() + ".jpg";
    File target = new File(folder, name);
    try (OutputStream out = new FileOutputStream(target)) {
        if (!bitmap.compress(Bitmap.CompressFormat.JPEG, JPEG_QUALITY, out)) throw new
IOException("JPEG encoding failed");
    } catch (IOException e) {
        target.delete();
        throw e;
    } finally {
        bitmap.recycle();
    }
    return name;
}
```

الشكل 19: نسخ الصورة المختارة بعد تصغيرها إلى مجلد خاص بالتطبيق

تُصغَّر الصورة إلى 1280 بكسل كحد أقصى مع تصحيح اتجاهها، وتُحفظ في `files/photos/`، فتبقى متاحة دون إنترنت حتى لو حُذفت من معرض الصور. وتتم المعالجة في خيط خلفي حتى لا تتجمد الواجهة، وتُحذف الصور غير المحفوظة عند الخروج من النموذج:

TreatedCaseFormActivity.java

```
/** تنسخ الصورة المختارة عبر PhotoStore في خيط خلفي حتى لا تتجمد الواجهة، ثم تعرض المعاينة. */
private void importPhoto(Uri uri) {
    importing = true; refreshPhoto();
    io.execute(() -> {
        String name = null;
        try { name = PhotoStore.importPhoto(getApplicationContext(), uri); } catch (Exception
ignored) { }
        String result = name;
        runOnUiThread(() -> {
            importing = false;
            if (isDestroyed()) { PhotoStore.delete(getApplicationContext(), result); return; }
            if (result == null) Toast.makeText(this, I18n.t(this, "photo_error"),
Toast.LENGTH_LONG).show();
            else { unsavedPhotos.add(result); photoName = result; }
            refreshPhoto();
        });
    });
}

/** عند الخروج دون حفظ تُحذف الصور المنسوخة غير المحفوظة حتى لا تبقى ملفات يتيمة، ثم يُغلق الخيط والقاعدة. */
@Override protected void onDestroy() {
    if (isFinishing()) for (String name : unsavedPhotos) PhotoStore.delete(this, name);
    io.shutdown();
    if (db != null) db.close();
    super.onDestroy();
}
```

الشكل 20: معالجة الصورة في خيط خلفي وحذف الصور غير المحفوظة

8. الاتصال المباشر والصلاحيات

تطلب الوظيفة تفعيل الاتصال المباشر من التطبيق. وضعنا طريقة الاتصال في الشاشة الأب `BaseActivity` حتى تستخدمها كل الشاشات:

BaseActivity.java

```
/** الرقم المنتظر الاتصال به ريثما يرد المستخدم على طلب صلاحية الاتصال. */
private String pendingCall;
/** طلب صلاحية CALL_PHONE: عند الموافقة اتصال مباشر، وعند الرفض فتح لوحة الاتصال. */
private final ActivityResultLauncher<String> callPermission = registerForActivityResult(new
ActivityResultContracts.RequestPermission(), granted -> {
    if (pendingCall != null) startCall(pendingCall, granted);
    pendingCall = null;
});

/** الاتصال المباشر بالرقم: إذا كانت صلاحية CALL_PHONE ممنوحة يتم الاتصال فوراً، وإلا تُطلب الصلاحية. */
protected void call(String phone) {
    if (ContextCompat.checkSelfPermission(this, Manifest.permission.CALL_PHONE) ==
PackageManager.PERMISSION_GRANTED) startCall(phone, true);
    else { pendingCall = phone; callPermission.launch(Manifest.permission.CALL_PHONE); }
}

/**
 * تنفيذ الاتصال: ACTION_CALL للاتصال المباشر، أو ACTION_DIAL لفتح لوحة الاتصال مع الرقم.
 * إذا لم يوجد تطبيق اتصال تظهر رسالة بدل الانهيار.
 */
private void startCall(String phone, boolean direct) {
    Uri uri = Uri.parse("tel:" + Uri.encode(phone));
    try {
        startActivity(new Intent(direct ? Intent.ACTION_CALL : Intent.ACTION_DIAL, uri));
    } catch (ActivityNotFoundException | SecurityException e) {
        try { startActivity(new Intent(Intent.ACTION_DIAL, uri)); }
        catch (ActivityNotFoundException none) { Toast.makeText(this, I18n.t(this, "no_dialer"),
Toast.LENGTH_LONG).show(); }
    }
}
```

الشكل 21: الاتصال المباشر مع طلب الصلاحية والرجوع إلى لوحة الاتصال

وفي أسفل شاشة الخطوات زر ثابت يعرض الأرقام المخزنة محلياً ويتصل بالرقم المختار مباشرة، كما في سيناريو الوظيفة:

EmergencyDetailActivity.java

```
/**
 * تعرض قائمة الأرقام المخزنة محلياً (العامة والخاصة)؛ الضغط على رقم يتصل به مباشرة،
 * وزر إدارة الأرقام يفتح شاشة أرقام الطوارئ.
 */
private void chooseNumberToCall() {
    List<String> labels = new ArrayList<>(), numbers = new ArrayList<>();
    Cursor c = db.getContacts();
    try { while (c.moveToNext()) { labels.add(I18n.t(this, c.getString(1)) + "  ·  " +
c.getString(2)); numbers.add(c.getString(2)); } } finally { c.close(); }
    new AlertDialog.Builder(this).setTitle(I18n.t(this, "call_emergency"))
            .setItems(labels.toArray(new String[0]), (d, which) -> call(numbers.get(which)))
            .setNeutralButton(I18n.t(this, "manage_contacts"), (d, w) -> startActivity(new
Intent(this, ContactsActivity.class)))
            .setNegativeButton(I18n.t(this, "cancel"), null).show();
}
```

الشكل 22: قائمة الاتصال السريع في شاشة الخطوات

AndroidManifest.xml

```
<!-- ميزة الاتصال غير إلزامية حتى يعمل التطبيق على الأجهزة اللوحية أيضاً -->
<uses-feature android:name="android.hardware.telephony" android:required="false" />
<!-- قراءة جهات اتصال الجهاز لاختيار رقم طوارئ منها -->
<uses-permission android:name="android.permission.READ_CONTACTS" />
<!-- الاتصال المباشر من التطبيق (عند الرفض تُفتح لوحة الاتصال) -->
<uses-permission android:name="android.permission.CALL_PHONE" />
<!-- لا توجد صلاحية INTERNET: التطبيق يعمل دون إنترنت بالكامل -->
```

الشكل 23: الصلاحيات في ملف AndroidManifest.xml

الصلاحية	الاستخدام	عند الرفض
READ_CONTACTS	اختيار رقم من جهات اتصال الجهاز	يُفتح منتقي النظام الذي لا يحتاج صلاحية
CALL_PHONE	الاتصال المباشر	تُفتح لوحة الاتصال مع الرقم جاهزاً

ملاحظة: نظام أندرويد نفسه يحوّل الاتصال بأرقام الطوارئ الرسمية (مثل 112) إلى لوحة الاتصال لمنع الاتصال الخاطئ، والاتصال المباشر يعمل مع الأرقام الخاصة.

9. معالجة الأخطاء والتحقق من المدخلات

الموضع	ما نتحقق منه
نموذج رقم الطوارئ	الاسم مطلوب؛ الرقم يقبل الأرقام و + ( ) - والمسافة فقط وبحد أدنى 3 أرقام؛ الخطأ يظهر بجانب الحقل والنافذة تبقى مفتوحة
نموذج الحالة المعالجة	الوصف والإجراءات مطلوبة؛ التاريخ بصيغة صحيحة وغير مستقبلي؛ حد أقصى لطول النصوص؛ لا حفظ أثناء معالجة الصورة
البحث	تهريب رموز LIKE، معاملات ?، ورجوع إلى LIKE إذا فشل FTS
الصور	صورة غير قابلة للقراءة أو ملف مفقود ← رسالة بدل الانهيار
الاتصال	رفض الصلاحية ← لوحة الاتصال؛ عدم وجود تطبيق اتصال ← رسالة
قاعدة البيانات	قيود NOT NULL و CHECK و UNIQUE والمفاتيح الأجنبية مفعّلة
تدوير الشاشة	حفظ كل حقول النموذج والصورة واسترجاعها

ContactsActivity.java

```
/**
 * نافذة إضافة أو تعديل رقم. التحقق: الاسم مطلوب، والرقم يقبل الأرقام و + ( ) - والمسافة فقط وبحد أدنى 3 أرقام.
 * عند الخطأ يظهر الخطأ بجانب الحقل وتبقى النافذة مفتوحة دون فقدان ما كُتب.
 */
private void edit(long id, String oldName, String oldPhone) {
    LinearLayout form = new LinearLayout(this);
    form.setOrientation(LinearLayout.VERTICAL);
    form.setPadding(Ui.dp(this, 20), Ui.dp(this, 4), Ui.dp(this, 20), 0);
    EditText name = Ui.input(this, I18n.t(this, "name"), false);
    name.setText(oldName);
    form.addView(name);
    EditText phone = Ui.input(this, I18n.t(this, "phone"), false);
    phone.setInputType(android.text.InputType.TYPE_CLASS_PHONE);
    LinearLayout.LayoutParams pp = Ui.lp(this, -1, -2);
    pp.topMargin = Ui.dp(this, 10);
    form.addView(phone, pp);
    phone.setText(oldPhone);
    Button choose = Ui.button(this, I18n.t(this, "choose_from_contacts"), false);
    LinearLayout.LayoutParams cp = Ui.lp(this, -1, 46);
    cp.topMargin = Ui.dp(this, 12);
    form.addView(choose, cp);
    choose.setOnClickListener(v -> chooseFromDeviceContacts());
    pendingName = name;
    pendingPhone = phone;
    AlertDialog dialog = new AlertDialog.Builder(this)
            .setTitle(id < 0 ? I18n.t(this, "add") : I18n.t(this, "edit"))
            .setView(form)
            .setNegativeButton(I18n.t(this, "cancel"), null)
            .setPositiveButton(I18n.t(this, "save"), null)
            .show();
    dialog.getButton(AlertDialog.BUTTON_POSITIVE).setOnClickListener(v -> {
        String n = name.getText().toString().trim();
        String p = phone.getText().toString().trim();
        boolean valid = true;
        if (n.isEmpty()) { name.setError(I18n.t(this, "field_required")); valid = false; }
        if (p.replaceAll("[+0-9() .-]", "").length() != 0 || p.replaceAll("\\D", "").length() < 3) {
phone.setError(I18n.t(this, "invalid_phone")); valid = false; }
        if (!valid) return;
        if (db.saveContact(id, n, p) < 0) { Toast.makeText(this, I18n.t(this, "save_failed"),
Toast.LENGTH_LONG).show(); return; }
        dialog.dismiss();
        load();
    });
}
```

الشكل 24: التحقق من رقم الطوارئ دون إغلاق النافذة

TreatedCaseFormActivity.java

```
/**
 * التحقق: الوصف والإجراءات مطلوبة، والتاريخ صحيح وغير مستقبلي؛ عند الخطأ يُعلَّم الحقل وتبقى الشاشة مفتوحة.
 * عند النجاح يُحفظ السجل وتُحذف الصورة القديمة إن استُبدلت.
 */
private void save() {
    String ds = description.getText().toString().trim(), dt = date.getText().toString().trim(), pr =
procedures.getText().toString().trim();
    boolean valid = true;
    if (ds.isEmpty()) { description.setError(I18n.t(this, "field_required")); valid = false; }
    Date parsed = parseDate(dt);
    if (parsed == null) { date.setError(I18n.t(this, "field_required")); valid = false; }
    else if (parsed.after(new Date())) { date.setError(I18n.t(this, "date_future")); valid = false;
}
    if (pr.isEmpty()) { procedures.setError(I18n.t(this, "field_required")); valid = false; }
    if (importing) { Toast.makeText(this, I18n.t(this, "photo_processing"),
Toast.LENGTH_SHORT).show(); return; }
    if (!valid) { Toast.makeText(this, I18n.t(this, "required"), Toast.LENGTH_SHORT).show(); return;
}
    if (db.saveTreatedCase(recordId, caseId, ds, dt, photoName, pr) < 0) { Toast.makeText(this,
I18n.t(this, "save_failed"), Toast.LENGTH_LONG).show(); return; }
    if (savedPhotoName != null && !savedPhotoName.equals(photoName)) PhotoStore.delete(this,
savedPhotoName);
    unsavedPhotos.remove(photoName);
    finish();
}

/** تحليل صارم لتاريخ بصيغة YYYY-MM-DD؛ تعيد null إذا كانت الصيغة أو التاريخ غير صحيحين. */
private Date parseDate(String value) {
    if (!value.matches("\\d{4}-\\d{2}-\\d{2}")) return null;
    try { return iso.parse(value); } catch (ParseException e) { return null; }
}
```

الشكل 25: التحقق من حقول الحالة المعالجة والتحليل الصارم للتاريخ

10. المشاكل البرمجية التي واجهتنا وكيف تغلبنا عليها

نستعرض هنا المشاكل الحقيقية التي واجهتنا أثناء التطوير، مع السبب الذي وصلنا إليه والحل الذي اعتمدناه وما تعلمناه.

10.1 انهيار التطبيق مع أول حرف في البحث

المشكلة: كان التطبيق يُغلق فور كتابة أي حرف في حقل البحث.

السبب: سجل الأخطاء أظهر `no such module: fts5`. كنا قد بنينا فهرس البحث بـ FTS5، لكن SQLite المدمج في أندرويد لا يحتوي FTS5.

الحل: انتقلنا إلى FTS4 الموجود في كل أجهزة أندرويد، ووضعنا البحث داخل `try/catch` مع رجوع تلقائي إلى `LIKE` وحده إذا لم يتوفر الفهرس.

DatabaseHelper.java

```
/**
 * تحذف وتعيد إنشاء جدول البحث النصي FTS4 وتملؤه من جدول الحالات.
 * إذا لم يدعم الجهاز FTS4 يُتجاهل الخطأ ويعمل البحث بـ LIKE فقط.
 */
private void rebuildFts(SQLiteDatabase db) {
    try {
        db.execSQL("DROP TABLE IF EXISTS emergencies_fts4");
        db.execSQL("CREATE VIRTUAL TABLE emergencies_fts4 USING fts4(title_en, title_ar,
symptoms_en, symptoms_ar, tokenize=unicode61)");
        db.execSQL("INSERT INTO emergencies_fts4(docid, title_en, title_ar, symptoms_en,
symptoms_ar) SELECT case_id, title_en, title_ar, symptoms_en, symptoms_ar FROM emergencies");
    } catch (SQLiteException e) {
    }
}
```

الشكل 26: إعادة بناء فهرس FTS4 مع تجاهل الخطأ على الأجهزة التي لا تدعمه

الدليل على نجاح الحل: البحث `sna ins` يجد "Snake or insect bite"، وهي نتيجة لا يستطيع `LIKE` إيجادها، أي أن `MATCH` يعمل فعلاً.

ما تعلمناه: لا نفترض أن ميزات SQLite على الحاسوب موجودة على الهاتف، ونجعل كل ميزة اختيارية قابلة للرجوع إلى بديل آمن.

10.2 البحث بالعربية لا يجد شيئاً

المشكلة الأولى: بعد الانتقال إلى FTS4 عمل البحث بالإنجليزية، لكن البحث ببادئة عربية مثل "الدغ" لم يجد نتيجة.

السبب: محلّل FTS4 الافتراضي (`simple`) لا يقسّم النص العربي بشكل صحيح.

الحل: أنشأنا الجدول بالمحلّل `tokenize=unicode61` الذي يدعم العربية.

المشكلة الثانية: ظهرت النصوص العربية في القاعدة الجاهزة رموزاً مشوهة (مثل ظ†ط²ظ‹ظ‹ظ‹

الحل: أعدنا كتابة السكربت بالنصوص الصحيحة، وأعدنا توليد القاعدة، وأضفنا ترحيلاً يصلح القاعدة عند المستخدمين الحاليين دون حذف بياناتهم.

ما تعلمناه: لا نعدّل الملفات التي تحتوي نصوصاً عربية إلا بأدوات تحفظ ترميز UTF-8، ونختبر البحث العربي بعد كل توليد للقاعدة.

10.3 شريط التنقل السفلي يميّز التبويب الخطأ

المشكلة: بعد التنقل بين التبويبات والرجوع، كان التبويب المميّز لا يطابق الشاشة الظاهرة.

السبب: بعد فحص كود مكتبة Material 1.10.0 وجدنا أنها تعكس قيمة الإرجاع من مستمع الاختيار: إرجاع `true` يجعل المكتبة تميّز العنصر المضغوط في شريط الشاشة المغادِرة.

الحل: نعيد `true` فقط عند الضغط على تبويب الشاشة الحالية، و `false` عند الانتقال لشاشة أخرى.

Ui.java

```
nav.setOnItemSelectedListener(item -> {
    int index = indexOfResId(resIds, item.getItemId());
    if (index < 0) return false;
    String id = ids[index];
    if (id.equals(selected)) return true;
    if (c instanceof Activity) {
        Intent intent = new Intent(c, destinations[index]);
        intent.addFlags(Intent.FLAG_ACTIVITY_REORDER_TO_FRONT);
        c.startActivity(intent);
        ((Activity) c).overridePendingTransition(0, 0);
    }
    return false;
});
```

الشكل 27: مستمع شريط التنقل بعد الإصلاح

ما تعلمناه: عند سلوك غريب في مكتبة جاهزة نقرأ كودها بدل التخمين.

10.4 تداخل المحتوى مع أشرطة النظام

المشكلة: مع استهداف API 36 يرسم أندرويد الواجهة خلف شريط الحالة وشريط التنقل إجبارياً، فاختفت عناوين الشاشات وشريط التبويبات خلفهما.

الحل: الدالة `Ui.edgeToEdge()` تجعل الأشرطة شفافة وتضبط لون أيقوناتها، و `Ui.screen()` تضيف مسافة شريط الحالة أعلى المحتوى ومسافة شريط التنقل أسفل التبويبات.

10.5 الوظيفة تطلب قاعدة جاهزة بينما كنا نملأ البيانات من الكود

المشكلة: النسخة الأولى كانت تُنشئ البيانات داخل `onCreate` ولا تشحن ملف قاعدة جاهزاً كما تطلب الوظيفة.

الحل: سكربت التوليد، ونسخ الملف من `assets` عند أول تشغيل، مع بقاء البيانات داخل الكود كنسخة احتياطية (القسم 4).

10.6 صورة الحالة كانت تُحفظ ولا تظهر

المشكلة: خزّنت النسخة الأولى رابط الصورة فقط (`content://`). لم تكن الصورة تظهر في أي مكان، وكانت تضيع إذا حذفها المستخدم من المعرض. وكان النموذج نافذة حوار تُغلق عند أي خطأ أو عند تدوير الشاشة، فيضيع ما كُتب.

الحل: الفئة `PhotoStore` (القسم 7)، ونقل النموذج إلى شاشة مستقلة فيها معاينة للصورة، وتحقق لا يغلق النموذج، وحفظ لكل الحقول عند تدوير الشاشة، وصورة مصغّرة في السجل.

10.7 نموذج رقم الطوارئ يمسح ما كتبه المستخدم

السبب: زر الحفظ في نافذة `AlertDialog` يغلقها دائماً، حتى عند وجود خطأ.

الحل: ربطنا مستمع زر الحفظ بعد إظهار النافذة عبر `getButton(BUTTON_POSITIVE)`، فلا تُغلق إلا بعد نجاح الحفظ، وتظهر رسالة الخطأ بجانب الحقل.

10.8 رموز البحث % و _ تطابق كل شيء

السبب: تمرير القيم كمعاملات يمنع حقن SQL، لكنه لا يلغي معنى رموز `LIKE` الخاصة.

الحل: الدالة `likePattern` تهرّب هذه الرموز ويُستخدم `ESCAPE` في الاستعلام.

10.9 الزوايا الدائرية شبه مربعة على الشاشات عالية الدقة

السبب: الدالة `rounded()` كانت تمرر قيمة dp مباشرة كبكسل.

الحل: تحويل نصف القطر وسماكة الإطار من dp إلى بكسل حسب كثافة الشاشة.

10.10 تثبيت ملف تطبيق قديم أثناء الاختبار

السبب: مع إصدار Gradle الجديد (AGP 9) يوجد ملف APK قديم في مجلد `intermediates` بينما الملف الصحيح في `outputs`.

الحل: التثبيت دائماً من `app/build/outputs/apk/debug/`.

10.11 المسافات بين العناصر في الاتجاه العربي

المشكلة: عند التقاط لقطات الشاشة باللغة العربية لاحظنا أن أرقام الخطوات ملتصقة بنص الخطوة، والصور المصغّرة ملتصقة بوصف السجل.

السبب: كنا نضع المسافات بـ `leftMargin` و `rightMargin`، وهي جهات ثابتة (يسار ويمين)، فعندما تنقلب الواجهة من اليمين لليسار تصبح المسافة في الجهة الخطأ.

الحل: استبدلنا كل المواضع (13 موضعاً) بـ `setMarginStart` و `setMarginEnd` التي تتبع اتجاه القراءة تلقائياً.

ما تعلمناه: في تطبيق ثنائي اللغة نستخدم دائماً البداية والنهاية (start/end) بدل اليسار واليمين، ونختبر كل شاشة باللغتين.

10.12 فجوات اكتشفناها بمراجعة الوظيفة سطراً بسطر

قبل التسليم راجعنا نص الوظيفة سطراً بسطر، فوجدنا: لا توجد حالة "كسور" مع أن الوظيفة تذكرها، ولا حالة بمستوى "طفيف"، وعنوان حالة اللدغات وترتيب خطواتها يختلفان عن مثال الوظيفة، والزر أسفل شاشة الخطوات يفتح شاشة الأرقام فقط بدل الاتصال السريع، والاتصال يفتح لوحة الاتصال فقط، واستعلامات الإحصاء تجمع دون ترتيب. أصلحنا كل ذلك في الإصدار 6 من القاعدة وفي الشاشات المعنية.

ما تعلمناه: مراجعة المتطلبات حرفياً في النهاية تكشف فجوات لا تظهر أثناء البرمجة.

11. الاختبار

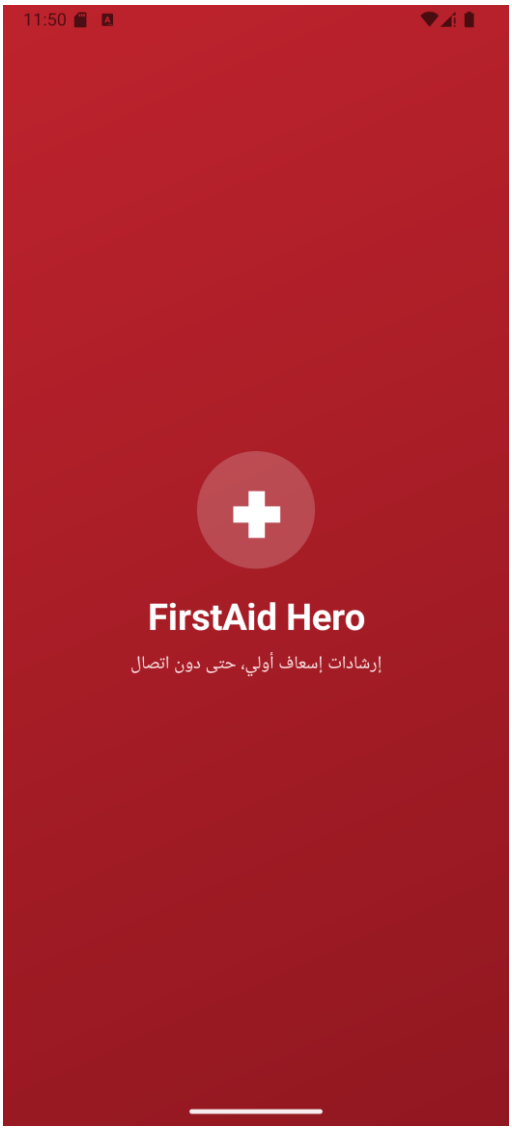
الاختبار	النتيجة المتوقعة
تشغيل التطبيق مع إيقاف الإنترنت	كل الشاشات تعمل
كتابة "لدغ" في البحث	تظهر "لدغات الأفاعي والحشرات"
كتابة `sna ins`	تظهر "Snake or insect bite" عبر `MATCH`
كتابة `%` في البحث	لا تظهر كل الحالات
فتح حالة	خطوات مرقمة بخط كبير وزر اتصال ثابت أسفل الشاشة
إضافة رقم برقم خاطئ مثل `abc`	رسالة خطأ بجانب الحقل والنافذة تبقى مفتوحة
محاولة تعديل رقم عام	غير ممكن
حالة معالجة دون وصف أو بتاريخ مستقبلي	رسالة خطأ ولا يُغلق النموذج
تسجيل حالة مع صورة ثم تدوير الشاشة	لا يضيع شيء
حذف سجل	يُحذف السجل وملف صورته
شاشة الإحصاءات	4 حرجة، 3 متوسطة، 1 طفيفة لحالات الدليل
ترقية من إصدار قديم للقاعدة	تبقى أرقام المستخدم وسجلاته

القسم الثاني: دليل المستخدم والمستثمر (User / Investor Guide)

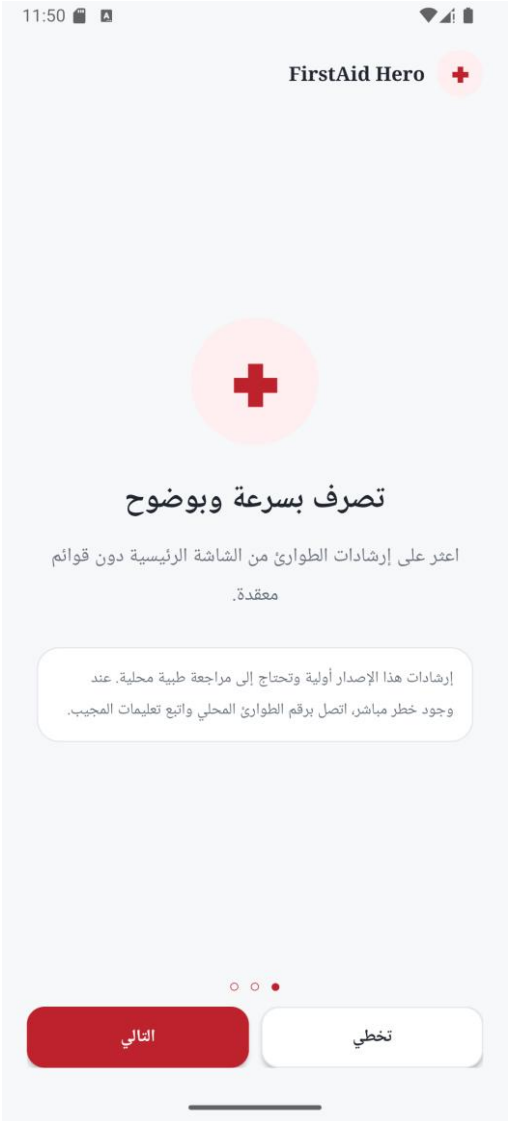
1. دليل التشغيل (User Manual)

1.1 التثبيت والتشغيل

1. يُثبَّت التطبيق على أي جهاز أندرويد 11 أو أحدث.
2. لا يحتاج التطبيق إنترنت ولا إنشاء حساب.
3. عند أول تشغيل تظهر شاشة البداية ثم ثلاث صفحات تعريفية يمكن تخطيها.



الشكل 28: شاشة البداية



الشكل 29: إحدى صفحات التعريف عند أول تشغيل

1.2 البحث عن حالة طارئة

1. في الشاشة الرئيسية شريط بحث كبير، وتحته قائمة بكل الحالات مرتبة من الأخطر إلى الأقل خطورة.
2. اكتب أي جزء من اسم الحالة أو أعراضها، بالعربية أو الإنجليزية، فتتحدث النتائج مع كل حرف.
3. اضغط على الحالة لفتح خطواتها.



الشكل 30: الشاشة الرئيسية وقائمة الحالات

مكان لقطة الشاشة(report/screenshots/04_search_ldg.png)

الشكل 31: البحث بكلمة "لدغ" يُظهر حالة لدغات الأفاعي والحشرات فوراً

1.3 اتباع خطوات الإسعاف والاتصال السريع

1. تعرض الشاشة عنوان الحالة ومستوى خطورتها والأعراض، ثم الخطوات مرقمة بخط كبير.
2. في أسفل الشاشة زر أحمر ثابت "اتصال سريع بأرقام الطوارئ".
3. اضغط الزر واختر الرقم فيتم الاتصال مباشرة.
4. زر "تسجيل هذه الحالة كمعالجة" يفتح نموذج السجل مع اختيار نوع الحالة تلقائياً.



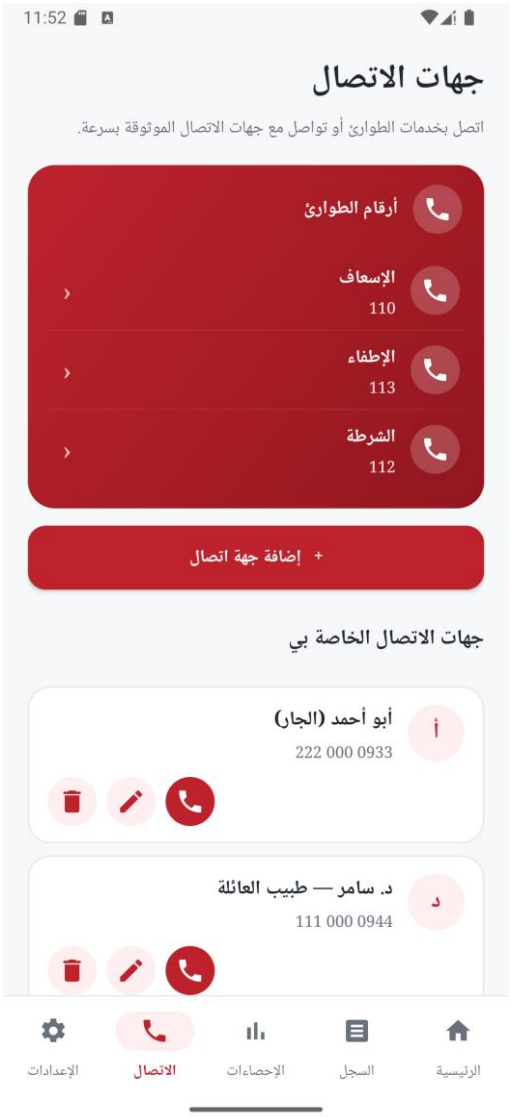
الشكل 32: شاشة خطوات الإسعاف مع زر الاتصال الثابت أسفل الشاشة



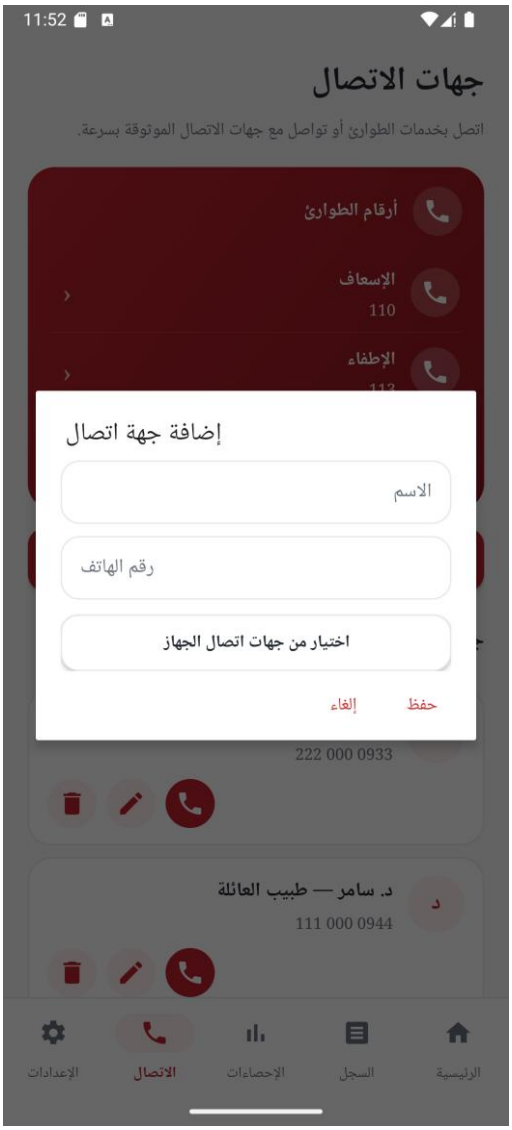
الشكل 33: قائمة الأرقام المخزنة للاتصال السريع

1.4 إضافة رقم طوارئ جديد

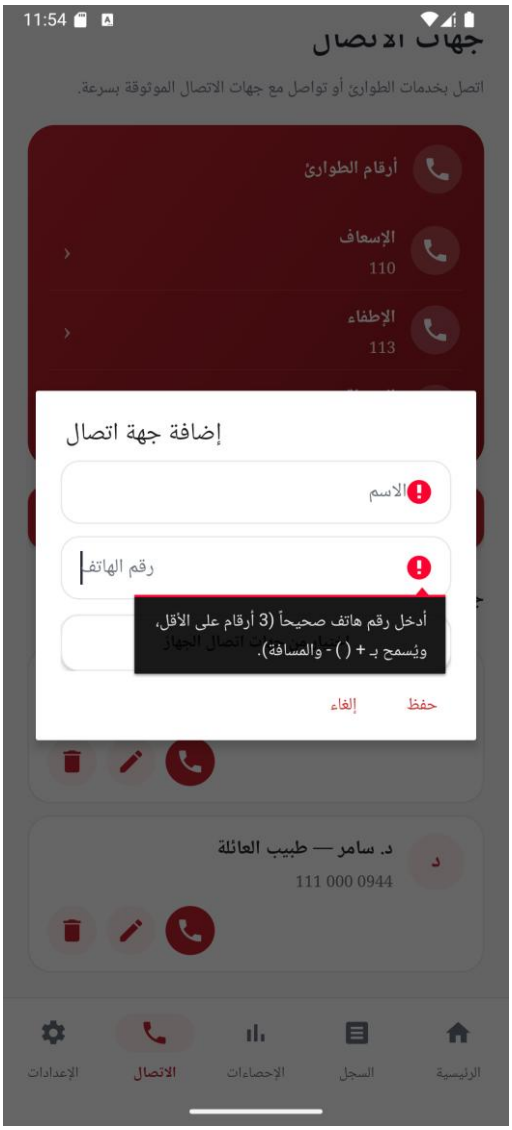
1. من شريط التنقل السفلي اختر "الاتصال".
2. في الأعلى الأرقام العامة (الشرطة، الإسعاف، الإطفاء)، والضغط على أي منها يتصل مباشرة.
3. اضغط "إضافة جهة اتصال"، واكتب الاسم والرقم (مثل رقم طبيب العائلة)، أو اضغط "اختيار من جهات اتصال الجهاز".
4. اضغط "حفظ". لكل رقم خاص أزرار للاتصال والتعديل والحذف.



الشكل 34: شاشة أرقام الطوارئ



الشكل 35: إضافة رقم طوارئ جديد



الشكل 36: رسالة خطأ عند إدخال رقم غير صحيح، والنافذة تبقى مفتوحة

1.5 تسجيل حالة معالجة

1. من "السجل" اضغط "تسجيل حالة معالجة"، أو من شاشة الخطوات اضغط "تسجيل هذه الحالة كمعالجة".
2. اختر نوع الحالة، واكتب الوصف، واختر التاريخ من التقويم، واكتب الإجراءات التي أُجريت.
3. اضغط "اختيار صورة" لإرفاق صورة للحالة، فتظهر معاينتها مباشرة.
4. اضغط "حفظ".



الشكل 37: نموذج تسجيل حالة معالجة مع معاينة الصورة



الشكل 38: سجل الحالات المعالجة مرتباً حسب التاريخ مع البحث

1.6 الإحصاءات والإعدادات

- شاشة "الإحصاءات" تعرض عدد الحالات حسب الخطورة، وعدد الحالات المعالجة حسب الخطورة وحسب الشهر.
- شاشة "الإعدادات" لاختيار اللغة (العربية أو English) والمظهر (حسب الجهاز، فاتح، داكن).



الشكل 39: شاشة الإحصاءات



الشكل 40: شاشة الإعدادات

2. فلسفة تجربة المستخدم (UI/UX Justification)

صممنا الواجهة لشخص متوتر يده ترتجف، في مكان قد يكون مظلماً أو مزدحماً، ولا وقت لديه لقراءة قوائم طويلة. كل قرار في التصميم يخدم هذه الحالة:

القرار	السبب
اللون الأحمر للبطاقة الرئيسية وأزرار الاتصال وأرقام الخطوات	الأحمر مرتبط في ذهن الناس بالطوارئ والإسعاف، ويجذب العين مباشرة إلى العناصر الأهم
الرمادي الداكن والأبيض لبقية الواجهة	تباين عالٍ يجعل النص مقروءاً، ولا يشتت الانتباه عن العناصر الحمراء
خط كبير للخطوات (19sp) وأرقام في دوائر	القراءة تحت الضغط أصعب؛ الخط الكبير والأرقام الواضحة تقلل الأخطاء وتحافظ على ترتيب الخطوات
شريط بحث كبير في أعلى الشاشة الرئيسية	المستخدم يصل إلى الحالة بكلمة واحدة دون تصفح قوائم
بحث بجزء من الكلمة وبالعربية والإنجليزية	المستخدم المتوتر يكتب كلمة ناقصة أو بلغة غير متوقعة
ترتيب النتائج حسب الخطورة	الحالات المهددة للحياة تظهر أولاً
زر اتصال ثابت أسفل شاشة الخطوات	لا يختفي مع التمرير، وفي متناول الإبهام؛ ضغطتان تكفيان للاتصال
عمل كامل دون إنترنت	الحوادث تقع غالباً في أماكن بلا تغطية (رحلات، طرق، مناطق نائية)
العربية مع اتجاه من اليمين لليسار	اللغة الأم للمستخدمين المستهدفين؛ الفهم أسرع وأدق
الوضع الداكن	قراءة مريحة ليلاً دون إبهار العين
رسائل الخطأ بجانب الحقل دون إغلاق النموذج	لا يخسر المستخدم ما كتبه بسبب خطأ صغير
تنبيه طبي واضح	التطبيق مرجع سريع لا يغني عن الاتصال بخدمات الطوارئ

3. جدوى المشروع (Value Proposition)

3.1 المشكلة

في الدقائق الأولى بعد الحادث يكون الشخص الموجود في المكان هو المسعف الوحيد. معظم الناس لا يتذكرون خطوات الإسعاف الصحيحة تحت الضغط، والبحث في الإنترنت بطيء أو غير ممكن في أماكن كثيرة، وقد يقود إلى معلومات غير موثوقة.

3.2 الحل

تطبيق صغير يعمل دون إنترنت، يعطي الخطوات الصحيحة مرتبة في ثوانٍ، ويربط المستخدم بأرقام الطوارئ بضغطتين، ويحفظ سجلاً للحالات التي عولجت مع صورها.

3.3 كيف ينقذ الأرواح

- **السرعة:** البحث يعيد النتيجة في أقل من ربع ملّي ثانية، والخطوات على بعد ضغطة واحدة.

- **الصحة:** خطوات مرتبة وواضحة تقلل الأخطاء الشائعة، مثل مص السم من اللدغة أو تقويم الكسر.
- **الاتصال المبكر:** زر الاتصال الثابت يقلل الوقت حتى طلب المساعدة المختصة.

3.4 الفئات المستهدفة

الفئة	القيمة
المدارس	مرجع إسعاف للمعلمين والمشرفين، وسجل للحالات التي عولجت مع صورها
السلامة المهنية في المصانع ومواقع البناء	استجابة أسرع لإصابات العمل وتوثيقها
الرحلات والأنشطة الخارجية والكشافة	يعمل في الأماكن التي لا تصلها شبكة
الأسر	مرجع منزلي لحوادث الأطفال والمسنين

3.5 الجدوى الاقتصادية

- **تكلفة تشغيل شبه معدومة:** لا يوجد خادم ولا قاعدة بيانات سحابية ولا اشتراكات، فكل البيانات على جهاز المستخدم. تكلفة إضافة مستخدم جديد صفر.
- **لا مخاطر على الخصوصية:** بيانات المستخدم (أرقامه وصور الحالات) لا تغادر جهازه.
- **سهولة التوسع:** إضافة حالات جديدة أو لغة جديدة تتم بتحديث ملف القاعدة الجاهزة ورفع رقم الإصدار، دون تغيير في بنية التطبيق.
- **نماذج تمويل ممكنة:** ترخيص نسخ مخصصة للمدارس والشركات (بأرقامها الداخلية وشعارها)، أو رعاية من جهات صحية ومنظمات إغاثة، أو تطبيق مجاني للجمهور بدعم من جهة السلامة العامة.

3.6 الحدود والمخاطر

- محتوى الإسعافات إرشادي عام ويجب أن تراجعه جهة طبية محلية قبل النشر العام.
- أرقام الطوارئ العامة يجب التأكد منها لكل بلد.
- التطبيق لا يغني عن خدمات الطوارئ، ويعرض ذلك بوضوح للمستخدم.

الخاتمة والتأمل

خرجنا من هذا المشروع بعدة دروس عملية:

- **المتطلب الحرفي مهم:** مراجعة نص الوظيفة سطراً بسطر في النهاية كشفت فجوات حقيقية (حالة الكسور، مستوى الخطورة الطفيف، الاتصال المباشر، زر الاتصال أسفل الشاشة) لم تظهر أثناء البرمجة.
- **قاعدة البيانات تعيش أطول من الكود:** كل تغيير في المخطط يحتاج ترحيلاً يحافظ على بيانات المستخدم، وطريقة تحديث المحتوى (UPSERT بدل الحذف) تؤثر على سلامة العلاقات.
- **الأجهزة ليست كالحاسوب:** ما يعمل في SQLite على الحاسوب (FTS5) قد لا يكون موجوداً على الهاتف، لذلك نبني دائماً طريقاً بديلاً آمناً.
- **النص العربي يحتاج عناية خاصة:** في الترميز (UTF-8) وفي البحث (محلّل `unicode61`) وفي العرض (الاتجاه من اليمين لليسار).
- **تجربة المستخدم في الطوارئ مختلفة:** كل ضغطة إضافية وكل نص صغير له ثمن، فصممنا كل شاشة لتختصر الطريق إلى الخطوة التالية.

تطويرات مستقبلية: مراجعة طبية للمحتوى وتوسيعه ليشمل حالات أكثر (الصرع، التسمم، ضربة الشمس)، وإضافة رسوم توضيحية للخطوات، والتقاط صورة الحالة بالكاميرا مباشرة، والانتقال إلى `RecyclerView` إذا كبر حجم البيانات، وكتابة اختبارات آلية لقاعدة البيانات والواجهات.

ملحق: ملفات المشروع

المسار	المحتوى
`app/src/main/java/com/firstaidhero/app/`	كود Java للتطبيق مع تعليقات عربية
`app/src/main/assets/databases/firstaid_hero.db`	قاعدة البيانات الجاهزة (الإصدار 6)
`app/src/main/AndroidManifest.xml`	الشاشات والصلاحيات
`tools/generate_assets_db.py`	سكربت توليد قاعدة البيانات الجاهزة
`docs/FULL_PROJECT_GUIDE_AR.md`	الدليل الشامل للمشروع بالعربية
`docs/CODE_AND_METHODS_AR.md`	شرح كل فئة وطريقة وقاعدة البيانات
`docs/CHECKLIST_AR.md`	مطابقة متطلبات الوظيفة سطراً بسطر